```
function [output,par_index,covar_index,defo_index] =
ps_model_definitions(info,model,Nifgs,h2ph,Btemp,Bdop,std_param)

% Function for the definition of deformation models. New models can
% easily be added (and only this file needs to be adapted). The
% first parameter of each model should be the topographic height.
%
% Input:  - info                'Npar' = number of parameters
%                               'Npar_max' = maximum Npar
%                               'model' = functional model
%         - model               model index
%                               1 = height + linear deformation
%                               2 = height + linear deformation + ...
%                                   periodic deformation
%                               3 = height + 2 linear deformations ...
%                                   (in case of event)
%                               4 = height + 2 linear deformations ...
%                                   + periodic deformation
%         - Nifgs               number of interferograms
%         - h2ph                height-to-phase factors
%         - Btemp               temporal baselines [year]
%         - Bdop                Doppler baselines [Hz]
%         - std_param           standard deviations for pseudo-
%                               observations
%
% Output: - output              number of parameters or functional model
%         - par_index           index of parameters
%         - covar_index         index of (co)variances
%         - defo_index          subindex of deformation parameters
%
% ------------------------------------------------------------------------
% File............: ps_model_definitions.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Author..........: Freek van Leijen
%                   Sami Samiei Esfahany
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
% v1.7.2.8, Sami Samiei Esfahany/Freek van Leijen
% - insert of double breakpoint model
%



% ------------------------------------------------------------------------
% Initialize
% ------------------------------------------------------------------------

global m2ph sat_vel althyp_index althyp_index2

Npar_max = 10;
first_defo_index = 4;
%do not forget to change these parameters when you change something

switch info
    case 'Npar_max'
```

```
      output = Npar_max;
      return
end

if length(model)>1
    error('Only one model can be parsed at the same time');
end

switch model

    %1 = height
    %2 = master atmosphere
    %3 = subpixel (azimuth)
    %4 = linear deformation 1
    %5 = linear deformation 2
    %6 = linear deformation 3
    %7 = quadratic deformation
    %8 = cubic deformation
    %9 = periodic deformation 1
    %10 = periodic deformation 2
  case 1 %linear for periodogram
    par_index = [1 4];
  case 2 %linear
    par_index = [1 2 4];
  case 3 %quadratic
    par_index = [1 2 4 7];
  case 4 %cubic
    par_index = [1 2 4 7 8];
  case 5 %linear+periodic
    par_index = [1 2 4 9 10];
  case 6 %quadratic+periodic
    par_index = [1 2 4 7 9 10];
  case 7 %cubic+periodic
    par_index = [1 2 4 7 8 9 10];
  case 8 %linear+subpixel
    par_index = [1 2 3 4];
  case 9 %quadratic+subpixel
    par_index = [1 2 3 4 7];
  case 10 %cubic+subpixel
    par_index = [1 2 3 4 7 8];
  case 11 %linear+periodic+subpixel
    par_index = [1 2 3 4 9 10];
  case 12 %quadratic+periodic+subpixel
    par_index = [1 2 3 4 7 9 10];
  case 13 %cubic+periodic+subpixel
    par_index = [1 2 3 4 7 8 9 10];
  case 14 %double linear
    par_index = [1 2 4 5];
  case 15 %double quadratic
    par_index = [1 2 4 5 7];
  case 16 %double cubic
    par_index = [1 2 4 5 7 8];
  case 17 %double linear+periodic
    par_index = [1 2 4 5 9 10];
  case 18 %double quadratic+periodic
    par_index = [1 2 4 5 7 9 10];
  case 19 %double cubic+periodic
    par_index = [1 2 4 5 7 8 9 10];
  case 20 %double linear+subpixel
    par_index = [1 2 3 4 5];
  case 21 %double quadratic+subpixel
    par_index = [1 2 3 4 5 7];
  case 22 %double cubic+subpixel
    par_index = [1 2 3 4 5 7 8];
```

```
  case 23 %double linear+periodic+subpixel
    par_index = [1 2 3 4 5 9 10];
  case 24 %double quadratic+periodic+subpixel
    par_index = [1 2 3 4 5 7 9 10];
  case 25 %double cubic+periodic+subpixel
    par_index = [1 2 3 4 5 7 8 9 10];
  case 26 %triple
    par_index = [1 2 4 5 6];
  case 27 %triple+subpixel
    par_index = [1 2 3 4 5 6];
  otherwise
    error('The model you specified is not implemented yet.');
end

%determine index for (co)variances
Npar = length(par_index);
covar_index = repmat(NaN,1,Npar*(Npar+1)/2);
count = 1;
for v = 1:Npar
    covar_index(count:count+Npar-v) = par_index(v:Npar)+sum(Npar_max-
par_index(v)+1:Npar_max-1);
    count = count+Npar-v+1;
end

%determine subindex for deformation parameters
defo_index = find(par_index>=first_defo_index);


switch info
  case 'Npar'
    output = length(par_index);
  case 'annotation'
    Npar = length(par_index);
    output = repmat({'not_used'},1,2*Npar_max);
    for v = 1:Npar
        par = par_index(v);
        switch par
          case 1
            output(1) = {'topo'};
            output(1+Npar_max) = {'m'};
          case 2
            output(2) = {'master_noise'};
            output(2+Npar_max) = {'m'};
          case 3
            output(3) = {'subpixel'};
            output(3+Npar_max) = {'m'};
          case 4
            output(4) = {'linear1'};
            output(4+Npar_max) = {'m/y'};
          case 5
            output(5) = {'linear2'};
            output(5+Npar_max) = {'m/y'};
          case 6
            output(6) = {'linear3'};          %%%sami
            output(6+Npar_max) = {'m/y'};     %%%sami
          case 7
            output(7) = {'quadratic'};
            output(7+Npar_max) = {'m/y^2'};
          case 8
            output(8) = {'cubic'};
            output(8+Npar_max) = {'m/y^3'};
          case 9
            output(9) = {'periodic1'};
            output(9+Npar_max) = {'-'};
```

```
        case 10
          output(10) = {'periodic2'};
          output(10+Npar_max) = {'-'};
      end
  end
case 'caxis'
  Npar = length(par_index);
  output = repmat(NaN,Npar_max,2);
  for v = 1:Npar
      par = par_index(v);
      switch par
        case 1
          output(1,:) = [-20 20];
        case 2
          output(2,:) = [-0.01 0.01];
        case 3
          output(3,:) = [-5 5];
        case 4
          output(4,:) = [-0.01 0.01];
        case 5
          output(5,:) = [-0.01 0.01];
        case 6
          output(6,:) = [-0.01 0.01];
        case 7
          output(7,:) = [-0.01 0.01];
        case 8
          output(8,:) = [-0.01 0.01];
        case 9
          output(9,:) = [-0.05 0.05];
        case 10
          output(10,:) = [-0.05 0.05];
      end
  end
case 'model'

  if model<=13 %model with or without breakpoint
      althyp_index = NaN;
      althyp_index2 = NaN;
  elseif model<=25
      althyp_index = althyp_index + 1;
      althyp_index2 = NaN;
  else                                                  %%%sami
      althyp_index = althyp_index + 1;
      althyp_index2 = althyp_index2 + 1;            %%%sami
  end
  bp1=althyp_index-1;    %%%sami
  bp2=althyp_index2-1;   %%%sami

  Npar = length(par_index);
  output = zeros(Nifgs+Npar,Npar);
  for v = 1:Npar
      par = par_index(v);
      switch par
        case 1
          output(1:Nifgs,v) = m2ph*h2ph;
          output(Nifgs+v,v) = std_param(1)^2;
        case 2
          output(1:Nifgs,v) = repmat(m2ph,Nifgs,1);
          output(Nifgs+v,v) = std_param(2)^2;
        case 3
          output(1:Nifgs,v) = (2*pi/sat_vel)*Bdop;
          output(Nifgs+v,v) = std_param(3)^2;
        case 4
          if isnan(althyp_index)
```

```
                output(1:Nifgs,v) = m2ph*Btemp;
            elseif isnan(althyp_index2)
                if Btemp(althyp_index-1)>0
                    output(1:Nifgs,v) = m2ph*[Btemp(1:althyp_index-
1);repmat(Btemp(althyp_index-1),Nifgs-althyp_index+1,1)];
                elseif Btemp(althyp_index-1)<0
                    output(1:Nifgs,v) = m2ph*[Btemp(1:althyp_index-2)-
repmat(Btemp(althyp_index-1),althyp_index-2,1);zeros(Nifgs-althyp_index+2,1)];
                end
             %%%%%%%%sami
            else

                if Btemp(althyp_index-1)>=0 & Btemp(althyp_index2-1)>=0
                    output(1:Nifgs,v) =
m2ph*[Btemp(1:bp1);repmat(Btemp(bp1),Nifgs-bp1,1)];
                elseif Btemp(althyp_index-1)<0 & Btemp(althyp_index2-1)>=0
                    output(1:Nifgs,v) = m2ph*[(Btemp(1:bp1)-
repmat(Btemp(bp1),bp1,1));repmat(0,Nifgs-bp1,1)];
                elseif Btemp(althyp_index-1)<0 & Btemp(althyp_index2-1)<0
                    output(1:Nifgs,v) = m2ph*[(Btemp(1:bp1)-
repmat(Btemp(bp1),bp1,1));repmat(0,Nifgs-bp1,1)];
                else
                    error('Bp2 should be larger than Bp1');
                end
            end
            %%%%%%%%%sami
            output(Nifgs+v,v) = std_param(4)^2;
          case 5
            if isnan(althyp_index)
                error('A wrong deformation model is specified');
            elseif isnan(althyp_index2)
                if Btemp(althyp_index-1)>0
                    output(1:Nifgs,v) = m2ph*[zeros(althyp_index-
1,1);Btemp(althyp_index:end)-repmat(Btemp(althyp_index-1),Nifgs-
althyp_index+1,1)];
                elseif Btemp(althyp_index-1)<0
                    output(1:Nifgs,v) = m2ph*[repmat(Btemp(althyp_index-
1),althyp_index-2,1);Btemp(althyp_index-1:end)];
                end
            else
                %%%%%%%%%%sami
                if Btemp(althyp_index-1)>=0 & Btemp(althyp_index2-1)>=0
                    output(1:Nifgs,v) = m2ph*[repmat(0,bp1,1);Btemp(bp1+1:bp2)-
repmat(Btemp(bp1),bp2-bp1,1);repmat(Btemp(bp2)-Btemp(bp1),Nifgs-bp2,1)];
                elseif Btemp(althyp_index-1)<0 & Btemp(althyp_index2-1)>=0
                    output(1:Nifgs,v) =
m2ph*[repmat(Btemp(bp1),bp1,1);Btemp(bp1+1:bp2);repmat(Btemp(bp2),Nifgs-bp2,1)];
                elseif Btemp(althyp_index-1)<0 & Btemp(althyp_index2-1)<0
                    output(1:Nifgs,v) = m2ph*[repmat(Btemp(bp1)-
Btemp(bp2),bp1,1);Btemp(bp1+1:bp2)-repmat(Btemp(bp2),bp2-bp1,1);repmat(0,Nifgs-
bp2,1)];
                else
                    error('Bp2 should be larger than Bp1');
                end
                %%%%%%%%%%%%sami
            end
            output(Nifgs+v,v) = std_param(4)^2;
          case 6
             %%%%%%%%%%sami
            if isnan(althyp_index)
               error('A wrong deformation model is specified');
            elseif isnan(althyp_index2)
               error('A wrong deformation model is specified');
            else
```

```
                if Btemp(althyp_index-1)>=0 & Btemp(althyp_index2-1)>=0
                    output(1:Nifgs,v) = m2ph*[repmat(0,bp2,1);Btemp(bp2+1:end)-
repmat(Btemp(bp2),Nifgs-bp2,1)];
                elseif Btemp(althyp_index-1)<0 & Btemp(althyp_index2-1)>=0
                    output(1:Nifgs,v) = m2ph*[repmat(0,bp2,1);Btemp(bp2+1:end)-
repmat(Btemp(bp2),Nifgs-bp2,1)];
                elseif Btemp(althyp_index-1)<0 & Btemp(althyp_index2-1)<0
                    output(1:Nifgs,v) =
m2ph*[repmat(Btemp(bp2),bp2,1);Btemp(bp2+1:end)];
                else
                    error('Bp2 should be larger than Bp1');
                end
                %%%%%%%%%%%%sami
            end
            output(Nifgs+v,v) = std_param(4)^2;
          case 7
            output(1:Nifgs,v) = m2ph*Btemp.^2;
            output(Nifgs+v,v) = std_param(5)^2;
          case 8
            output(1:Nifgs,v) = m2ph*Btemp.^3;
            output(Nifgs+v,v) = std_param(6)^2;
          case 9
            output(1:Nifgs,v) = m2ph*sin(2*pi*Btemp);
            output(Nifgs+v,v) = std_param(7)^2;
          case 10
            output(1:Nifgs,v) = m2ph*(cos(2*pi*Btemp)-1);
            output(Nifgs+v,v) = std_param(7)^2;
        end
    end
end
```